

PORTADA

Manual Oficial de Usuario miClub Gestion

Sistema Integral de Gestion Operativa

Version del documento: 1.0 Capitulo 1 Inicio

Fecha de generacion: 30/06/2026

Estado: Version Estable

Logo: apps/web/public/logo/miClub - Logo trans.png

Actualizacion post-admin: 06/08/2026. El capitulo operativo vigente de Administracion se incorpora al final de este documento. Los artefactos HTML/PDF deben regenerarse antes de distribucion externa.

INDICE

Capitulo 0 Presentacion

Capitulo 1 Inicio

Capitulo 2 Economia Club

Capitulo 3 Espacio Fitness

Capitulo 4 Salon

Capitulo 5 Aula

Capitulo 6 Local 1

Capitulo 7 Cantina

Capitulo 8 CRM

Capitulo 9 Migracion

Capitulo 10 Administracion

Capitulo 11 Arquitectura Tecnica

Apendices

CAPITULO 1 INICIO

Alcance auditado. La pantalla Inicio esta implementada por HomeModule, useHomeDashboard, HomeMetricCards, RecentMovements y SectorDistribution. Consume endpoints raiz legacy compatibles: /summary, /members, /debtors, /sync-status, /club-finance-summary y /sector-operational-summary. En el estado productivo declarado, PostgreSQL es la fuente oficial; Google Sheets fue retirado y solo se conserva como antecedente historico no ejecutable.

1. Panel Operativo

Que representa: cabecera ejecutiva del Inicio con titulo Panel operativo de miClub, estado de sincronizacion, fecha de ultima sincronizacion y boton Sincronizar.

Estado: el frontend muestra Con advertencias si /sync-status devuelve error; muestra Google Sheets conectado si la fuente fuese google_sheets; para cualquier otro caso muestra Datos mock/locales. Inconsistencia: para source=postgres no existe etiqueta especifica y el frontend cae en Datos mock/locales, aunque el backend productivo responde source=postgres. Impacto: puede confundir al usuario sobre el origen oficial. Solucion: agregar etiqueta PostgreSQL conectado.

Ultima sincronizacion: se formatea en es-AR desde lastSyncAt. En PostgreSQL, /sync-status ejecuta health check; si es correcto actualiza lastSyncAt con la fecha/hora actual.

Boton Sincronizar: vuelve a ejecutar loadHome y recarga en paralelo todos los endpoints del Inicio. No importa datos ni escribe en PostgreSQL; solo refresca la vista.

Origen de datos: DATA_SOURCE=postgres activa PostgreSQL mediante shouldUsePostgresDataSource. Google Sheets no es backend ni proceso de migracion disponible: su API, credenciales y dependencia 'googleapis' fueron retiradas.

Tablas/servicios: /sync-status usa getPostgresHealth y validatePostgresEnv. Los datos funcionales se construyen desde people, enrollments, activities, sectors, movements, operational_balances y sheet_metric_snapshots.

2. Resumen Financiero

Endpoint: GET /club-finance-summary. Servicio: getPostgresClubFinanceSummary. Consultas principales: miclub.v_dashboard_basic, miclub.operational_balances, miclub.v_sector_settlement_balances y miclub.v_movements_enriched.

Liquidez: valor destacado. En la vista SQL se calcula como capital sin dolares + saldos positivos a liquidar + dolares valorizados por tipo de cambio del ultimo operational_balances. En el servicio se

toma de `operational_balances` mas reciente o de `v_dashboard_basic`. Interpretacion: capacidad economica operativa actual del club.

Caja: suma neta de movimientos administrativos completados con `payment_method` efectivo: ingresos/capital suman, egresos restan.

Banco: suma neta de movimientos administrativos completados con `payment_method` transferencia: ingresos/capital suman, egresos restan.

Dolares: suma neta de movimientos administrativos completados cuya categoria normalizada es DOLARES: ingresos/capital suman y egresos restan. Se presenta como USD entero redondeado.

Movimientos incluidos: movimientos de Administracion completados para saldos reales; capital, ingresos y egresos segun tipo; metodo de pago efectivo/transferencia; categoria Dolares para USD.

Movimientos excluidos: pendientes, cancelados/no completados para caja/banco/dolares; movimientos fuera de Administracion para saldos administrativos; importes sin estado o categoria que no cumplen las reglas.

Ajustes manuales: `operational_balances` se ordena por `cutoff_date` y `created_at` descendente; el registro mas reciente prevalece como foto operativa, incluyendo `source_payload` con tipo de cambio historico.

3. Saldos Operativos

Endpoint: `GET /club-finance-summary`. Servicio: `getPostgresClubFinanceSummary`. Contrato compartido: `ClubOperationsSummary`.

Cuotas a Cobrar: suma de `receivable_fee` de inscripciones cuyo estado efectivo es adeudando. La cuota se normaliza si llego inflada por mil: valores enteros $\geq 1.000.000$ divisibles por 1.000 se dividen por 1.000. Se excluyen cuotas ≤ 0 , abandonados y cancelados. Comision aplicada: Fitness 50%; Salon 0%; Aula usa `club_commission_percent` acotado entre 0 y 1; otros sectores 0.

Saldos Pendientes: $\text{pendingNetBalance} = \text{pendingIncome} - \text{pendingExpenses}$. El SQL de la vista suma ademas `futureReceivableFeesUntilMonthEnd`, pero el servicio usa un fallback directo sobre `v_movements_enriched` filtrando `source_payload.sheet = ADMINISTRACION` y estado operativo PENDIENTE o estado financiero pendiente; por eso el valor de la pantalla es neto de ingresos pendientes menos egresos pendientes administrativos.

Saldos a Liquidar: suma de saldos positivos a liquidar por sector. El servicio toma `v_sector_settlement_balances` con `settlement_balance > 0` y complementa snapshots historicos `fitness/salon/aula/local1`. Se muestra con signo negativo porque representa obligacion.

Saldo Proyectado: formula critica del backend: $\text{Liquidez} + \text{Cuotas a cobrar} + \text{Saldos pendientes} - \text{Saldos a pagar}$. Cambia al modificar movimientos pendientes, pagos/estados de inscripciones, saldos de liquidacion o liquidez.

Registros que afectan: enrollments.fee_amount/status/due_date/activity_id;
activities.club_commission_percent/sector_id; sectors.code/name;
movements.amount/type/status/financial_status/source_payload; operational_balances;
sheet_metric_snapshots.

4. Ingresos por Sector

Endpoint /club-finance-summary. SQL: v_movements_enriched filtrada por movement_type=INGRESOS y operational_status=COMPLETADO, agrupada por sector_name con Sin datos como fallback. Se ordena por monto descendente y nombre ascendente, limitado a 4 filas. La pantalla marca con estrella el sector principal. No cuenta egresos, pendientes ni movimientos no completados.

5. Egresos por Sector

Mismo endpoint y servicio. Filtra movement_type=EGRESOS y operational_status=COMPLETADO, agrupa por sector_name, ordena por monto descendente y limita a 4. El primer egreso se marca con señal visual negativa. No incluye ingresos ni pendientes.

6. Inscriptos

Endpoints: /summary, /members y /debtors. Servicios: getPostgresSummary, getPostgresMembers, getPostgresDebtors.

Total: cantidad de filas actuales de v_current_enrollments mapeadas como miembros.

Activos: inscripciones cuyo estado normalizado no es abandonado ni cancelado.

Al día: estado normalizado al_día. Nuevo inscripto con due_date vigente puede convertirse en Al día durante el mapeo.

Adeudando: estado normalizado adeudando. Nuevo inscripto vencido puede convertirse en Adeudando.

Abandonados: estado abandonado.

Cancelados: estado cancelado.

Cuota promedio: promedio simple de cuotas positivas de miembros activos cargados en frontend.

Adeudados por actividad: se filtran miembros deudores, se agrupan por actividad o modalidad, se ordenan por cantidad descendente y nombre; se muestran las primeras 3 y un contador de actividades restantes.

7. Inscriptos por actividad

Se calcula en frontend con miembros activos: excluye abandonados y cancelados, agrupa por actividad/modalidad, ordena por cantidad descendente y nombre, muestra las primeras 6 y destaca la primera. No usa un endpoint específico adicional: depende de /members.

8. Distribucion Operativa por Sector

Endpoint: /sector-operational-summary. Servicio: getPostgresSectorOperationalSummary.

Espacio Fitness: total de inscriptos, activos, adeudados, monto adeudado, rentabilidad total, rentabilidad mensual y saldo a liquidar. Rentabilidad y saldo provienen de sheet_metric_snapshots; si faltan, se marca pendiente de calculo. Adeudados suman cuotas de miembros Fitness con estado adeudando.

Local 1: rentabilidad total, rentabilidad mensual, saldo a liquidar, total ventas, ventas ultimos 30 días e ingreso destacado. Ventas relevantes son ingresos completados del sector Local 1 con categoria comision o ventas. Ingreso destacado es el mayor monto y, ante empate operativo, el mas reciente.

Salon: total, activos, rentabilidad total/mensual, actividad mas popular y menos popular.

Popularidad usa inscripciones no abandonadas/no canceladas agrupadas por actividad, ordenadas por cantidad.

Aula: total, activos, rentabilidad total/mensual, comision promedio y actividad mas popular.

Comision promedio proviene de snapshot aula.average_commission.

Cantina: kiosco, bebidas, CMV y rentabilidad total. Formula: ingresos kiosco + ingresos bebidas - egresos bebidas. Puede usar snapshots y fallback consultado.

CRM: total de miembros, activos, deudores y monto adeudado general. Es una tarjeta de vision transversal de cobranzas.

Interpretacion Operativa

Primero mirar Liquidez y Saldo proyectado: si ambos bajan, existe riesgo financiero. Luego revisar Saldos a Liquidar para entender obligaciones proximas. Despues analizar Cuotas a cobrar y Adeudados por actividad para detectar morosidad concentrada. Ingresos y egresos por sector muestran donde se genera o consume dinero. Las tarjetas sectoriales permiten identificar que unidad crece, cual pierde rentabilidad y donde conviene accionar. Un crecimiento sano combina mas activos, ingresos completados y rentabilidad mensual positiva. Una alerta de morosidad aparece cuando aumentan

Adeudando, Monto adeudado o Cuotas a cobrar sin mejora en liquidez. Una caída de ingresos se ve cuando el ranking de ingresos baja o cuando rentabilidad mensual queda pendiente, negativa o menor al patron esperado.

Buenas practicas

Actualizar Inicio antes de tomar decisiones. Leer saldos proyectados como estimacion operativa, no como extracto bancario. Validar pagos pendientes antes de liquidar sectores. Revisar actividades con deudores altos. Usar tarjetas sectoriales para priorizar seguimiento.

Errores comunes

Confundir Saldos a Liquidar con dinero disponible. Leer Cuotas a cobrar como efectivo confirmado. Comparar Dolares con pesos sin tipo de cambio. Tratar Google Sheets como backend actual. Ignorar metricas pendiente de calculo.

Recomendaciones

Agregar etiqueta PostgreSQL conectado. Revisar diferencias entre v_dashboard_basic y fallback del servicio en Saldos Pendientes. Consolidar snapshots criticos o reemplazarlos por vistas PostgreSQL autoritativas. Mantener pruebas sobre saldo proyectado, receivables y estados.

Inconsistencias detectadas

1. Etiqueta de origen en frontend: source=postgres se muestra como Datos mock/locales. Impacto alto de UX; solucion: mapear postgres a PostgreSQL conectado.
2. Saldos Pendientes: v_dashboard_basic incluye cuotas futuras del mes en pending_net_balance, pero getPostgresClubFinanceSummary prioriza fallback directo sin cuotas futuras. Impacto: posible diferencia entre endpoint /api/dashboard/basic y pantalla Inicio. Solucion: unificar formula.
3. Metricas sectoriales: varias rentabilidades y saldos dependen de sheet_metric_snapshots importados historicamente. Impacto: cobertura parcial si falta snapshot. Solucion: vistas PostgreSQL completas por sector.
4. Rutas legacy raiz: el frontend consume /summary, /members, /debtors y no rutas /api. Impacto: contrato heredado; solucion: documentar y migrar con compatibilidad.

La fila analitica anual de Economia Club utiliza 'GET /api/economy/yearly-breakdown' con fecha de referencia opcional '?asOf=AAAA-MM-DD' y muestra los graficos Ingresos Operativos por Categoria y Gastos por Tipo con una ventana movil interanual inclusiva en zona 'America/Argentina/Buenos_Aires': desde el mismo mes del ano anterior hasta el mes actual, ambos extremos incluidos. Por esta regla aprobada se devuelven 13 puntos mensuales, con labels que incluyen mes y ano, claves 'AAAA-MM' para no mezclar meses iguales de anos distintos y valores cero para meses sin movimientos. El backend filtra movimientos consolidados con 'operational_status = COMPLETADO', usa 'movement_date >= fromMonth' y 'movement_date < toExclusive', excluye pendientes/cancelados/anulados por estado operativo y no modifica datos historicos.

Categorias operativas para ingresos y gastos operativos: INSCRIPCION, CUOTA, TURNOS, COMISION, ALQUILER, EVENTOS, VENTAS, CLASES, CURSOS, KIOSCO y BEBIDAS. El grafico de ingresos operativos incluye solo 'movement_type = INGRESOS', excluye CAPITAL y publica unicamente categorias con total interanual mayor que cero.

El clasificador unico de gastos aplica prioridad DEBT, SERVICES, TAXES, OPERATING, NON_OPERATING y UNCLASSIFIED. Las categorias no operativas son PUBLICIDAD, SALARIOS, MANTENIM., DEPOSITOS, EXTRACCIONES, DOLARES, REPARACIONES, VIATICOS, GANANCIA, PERDIDA, CMV, SEGUROS, LIMPIEZA, LIBRERIA y OTROS. Deudas/Pasivos usa DEUDA y DEUDAS; Servicios usa LUZ, AGUA e INTERNET; Impuestos usa IMPUESTO e IMPUESTOS.

Convencion de signos del grafico Gastos por Tipo: para Gastos Operativos y Gastos No Operativos se suman unicamente EGRESOS; para Deudas/Pasivos, Servicios e Impuestos se calcula EGRESOS - INGRESOS y se preserva el signo sin 'ABS[]'. Un valor negativo representa ingreso/reintegro neto superior al egreso del mes. Los movimientos no clasificados se reportan en metadata y en el script 'npm run audit:economy-yearly-breakdown -- --asOf=AAAA-MM-DD' sin reclasificarlos como OTROS.

CAPITULO 9 MIGRACION XLSX [VIGENTE]

La importacion soportada recibe un archivo '.xlsx' mediante 'POST /api/migration'. Es independiente de Google Sheets: no consulta Google Sheets API, no requiere credenciales 'GOOGLE_*' y no utiliza la dependencia 'googleapis'. Se debe ejecutar primero el dry-run, revisar errores y conservar el hash y reporte del lote antes de persistir. La guia operativa vigente es 'docs/import-xlsx.md'.

Los antiguos procedimientos que leian rangos de una planilla Google o archivaban inscripciones de ese importador estan en 'docs/history/' y no deben ejecutarse en instalaciones nuevas. XLSX continua soportado aunque Google Sheets haya sido retirado.

CAPITULO 10 ADMINISTRACION [ACTUALIZACION POST-ADMIN]

Acceso y navegacion

Ingresar con una membresia activa que tenga 'administration.view' y elegir **Administracion** en la navegacion. Si aparece No autorizado, solicitar el permiso a un administrador; cambiar la URL o enviar otro 'clubId' no cambia el club de la sesion. El panel muestra estados de carga, vacio y error con opcion de reintentar.

Lectura del panel

Las tarjetas superiores resumen inscripciones activas, capacidad, trabajadores y actividades, con comparaciones cuando existe historial suficiente. Revisar la fecha y el mensaje de disponibilidad antes de interpretar una variacion. Los rankings y tendencias son indicadores operativos, no un cierre contable.

En ****Sectores****, seleccionar una fila para consultar responsable, capacidad, actividades y movimientos relacionados. Los sectores de sistema estan protegidos. En ****Actividades****, seleccionar una fila para consultar configuracion, inscriptos, movimientos asociados mediante 'activity_id' y auditoria. En ****Trabajadores****, abrir la ficha para consultar relacion laboral, acceso, permisos y actividades. Estas tres fichas son de solo lectura.

Si Trabajadores muestra una advertencia de fuente legacy, salario y fecha de ingreso pueden no estar disponibles; no completar esos datos por inferencia. ****Actualizar**** vuelve a consultar PostgreSQL y no importa datos externos.

Movimientos e inscripciones

****Cargar Movimiento**** solo se habilita con 'movements.create'. Completar fecha, ingreso/egreso, categoria, sector, actividad si corresponde, concepto, contraparte, importe, medio de pago y estado inicial. El formulario envia una clave de idempotencia para evitar duplicados ante reintentos. Confirmar el resultado antes de volver a cargarlo. Un movimiento conciliado o aplicado a un pago no puede editarse; editar requiere 'movements.edit' y anular exige motivo y 'movements.cancel'.

****Cargar Inscripcion**** requiere 'enrollments.create'. Elegir una persona y una actividad activa que genere inscripciones, indicar cuota, estado y fechas. El sistema rechaza referencias de otro club y una inscripcion activa duplicada. Las futuras operaciones de edicion y cancelacion usaran 'enrollments.edit' y 'enrollments.cancel'. Los permisos amplios 'finance:write' y 'club:manage' solo se aceptan como transicion hasta el 2026-11-06; las membresias nuevas deben recibir siempre los permisos granulares.

Tareas y solicitudes

En ****Tareas****, crear titulo, descripcion opcional y vencimiento; luego cambiar entre Pendiente, En curso, Completada o Cancelada, o archivar. Si otra sesion modifiko la tarea, recargar antes de reintentar. La etiqueta Vencida se calcula para tareas pendientes cuya fecha ya paso.

Las solicitudes se pueden consultar con 'requests.view'. Aprobar o rechazar requiere el permiso correspondiente; una decision ya tomada no se repite y los tipos sin handler seguro no se ejecutan. Registrar un motivo claro aun cuando sea opcional.

Acciones todavia no disponibles

Gestionar categorias, trabajadores, cuotas y socios desde las tarjetas rapidas todavia no abre un flujo completo. Reservas y Membresias aparecen como ****Proximamente**** hasta definir modelo, disponibilidad, pagos y cancelaciones. No usar SQL manual para sustituir esas funciones.

Buenas practicas y soporte

Actualizar el panel antes de decidir, no compartir sesiones, no modificar el tenant desde herramientas del navegador y conservar el 'requestId' de cualquier error. Ante una falla de escritura, no repetir compulsivamente: verificar primero si la operacion quedo registrada. El indice vigente para localizar las fuentes tecnicas, procedimientos de rollback y evidencia exigida es 'docs/pre-reset-readiness.md'; el manual no afirma el estado aplicado de un entorno.